

UNIVERSIDAD PERUANA
CAYETANO HEREDIA



FACULTAD DE CIENCIAS E INGENIERÍA

“ALBERTO CAZORLA TALLERÍ”

Departamento Académico de Ciencias Exactas - Sección Ingeniería

INFORME 6

CURSO

SISTEMAS OPERATIVOS

INTEGRANTES

- Chara Franco, Johan Daymar
- Huancas Bereche, Erick Emilio
- Pellanne Aro, Carlos Alfredo
- Reyes Garcia Alvaro Pablo
- Rivera Garayar, Matias Gabriel
- Romero Chac, Luis Antonio

DOCENTE

Dr. Roberto Eduardo Burga Montañez

GRUPO

SO2 - G2

Informe Detallado: Gestión de Memoria mediante Segmentación y Paginación

Nota sobre la extensión: El presente informe desarrolla exhaustivamente todos los conceptos teóricos y prácticos mencionados en la fuente proporcionada para ofrecer la máxima profundidad requerida, estructurando la transcripción educativa en un formato formal y académico.

1. Introducción a la Segmentación

La segmentación es un esquema de gestión de memoria que, de manera similar a la paginación, permite disponer del proceso en la memoria de forma no consecutiva o continua. A diferencia de la paginación, la segmentación interpreta la memoria como un conjunto de segmentos de tamaño variable. Los segmentos no tienen ninguna ordenación específica dentro de la memoria. Este método permite a los sistemas disponer distintas partes de un programa en diferentes lugares o "huecos" de la memoria, mejorando el aprovechamiento y la eficiencia general del sistema.

2. Espacio Lógico y Uso Práctico

El espacio lógico de direcciones se compone de una colección de segmentos, donde cada uno de ellos posee un nombre y una longitud específica. Para especificar una dirección de memoria, el usuario o el programador debe proporcionar obligatoriamente dos valores: el nombre (o número) del segmento y el desplazamiento dentro de dicho segmento.

El uso de la segmentación facilita la vida al programador ya que permite disponer de áreas de memoria especializadas, tales como el área de código, el área de datos y la pila (stack), y ubicarlas en distintas partes del mapa de memoria. Normalmente, la lógica de programación dicta que un programa no mezcle variables sueltas en medio de un lazo de código, sino que mantenga una separación estructurada. Por este motivo, los compiladores modernos automatizan este proceso: ubican la sección de datos en su respectivo segmento de datos, el código ejecutable en un segmento de código y la pila en un segmento dedicado, aplicando la misma regla para las bibliotecas incluidas en el programa.

3. Direccionamiento y Tablas de Segmentos

En la segmentación, la dirección lógica proporcionada está compuesta por un número de segmento (o selector) y un desplazamiento. El número de segmento es utilizado por el hardware como índice para buscar dentro de una tabla de segmentos.

Esta tabla de segmentos proporciona dos valores fundamentales para cada entrada: la dirección base y el límite. La dirección base indica dónde comienza exactamente el segmento

dentro de la memoria física. Por otro lado, el **límite** indica el tamaño o tope del segmento, lo que sirve al sistema operativo para dimensionarlo.

El desplazamiento solicitado dentro de la dirección lógica debe estar estrictamente comprendido entre 0 y el límite del segmento. El sistema operativo, a través del hardware, compara el límite con el desplazamiento que el programa intenta acceder. Si el programa intenta acceder por fuera de este límite (por ejemplo, al intentar escribir más allá del tamaño establecido del segmento), el hardware genera inmediatamente una interrupción por error de direccionamiento.

Dicha interrupción debe ser atendida por el sistema operativo, el cual determinará si fue un fallo crítico de direccionamiento, si se trata del acceso a una parte de la memoria compartida, o si simplemente el segmento reside en el disco y dicha interrupción obliga a traerlo a memoria principal. Si el acceso es válido, la dirección física final se obtiene sumando la dirección base (extraída de la tabla) con el desplazamiento.

4. Selectores y Descriptores de Segmento

El número de segmento en la dirección lógica actúa como un **selector de segmento**. Este selector es un registro de 16 bits que tiene la tarea de definir en qué tabla se buscará el segmento. Existen bits específicos en el selector (el Indicador de Tabla o TI) que determinan si la búsqueda debe realizarse en la Tabla de Descriptores Globales (GDT) o en la Tabla de Descriptores Locales (LDT). El selector de 16 bits incluye también el Nivel de Privilegio Requerido (RPL).

Al acceder a la tabla indicada por el selector, se obtiene un **descriptor de segmento**. En procesadores Intel x86 o entornos como Linux, un descriptor es una estructura compleja de 64 bits (o 8 bytes). El descriptor incluye los 32 bits de la dirección base y 20 bits de límite. Adicionalmente, el descriptor define de forma administrativa de qué tipo de segmento se trata (código, datos, o sistema) y estipula sus niveles de acceso (por ejemplo, si permite lectura, escritura, etc.).

5. Protección y Privilegios de Seguridad

La segmentación, además de organizar la memoria, provee un fuerte mecanismo de protección. Por ejemplo, un área de código se puede definir dentro de su descriptor estrictamente como "solo lectura". Si un sistema permitiera escribir en un área de código desprotegida, un proceso malicioso podría tomar el control y escribir un virus (como en algunos ataques conocidos). Al definir el segmento de código como solo lectura, el procesador bloquea el intento de sobreescritura.

De manera similar, el área de datos puede configurarse individualmente para permitir tanto operaciones de lectura como de escritura, según se requiera. Cabe resaltar que cualquier intento de acceso o modificación a estas tablas de segmentos está resguardado: los registros administrativos requieren que el proceso esté en nivel de privilegio 0 para ser manipulados, estando fuertemente protegidos por el sistema operativo.

6. Registros Internos de la CPU

Para la gestión de segmentos, el procesador cuenta con registros de segmentos internos tales como: CS (Código), DS (Datos), SS (Pila), ES, FS y GS. Estos registros actúan como punteros hacia las entradas en las tablas de descriptores. Cuando el usuario o un programa carga uno de estos registros con un selector, el sistema interpreta que se requiere acceder a un segmento específico.

Para que el hardware sepa dónde encontrar las tablas de descriptores en la memoria general, la arquitectura utiliza registros especiales llamados GDTR y LDTR. Estos apuntan a las direcciones base de la memoria física donde comienzan las tablas de descriptores globales (GDT) y locales (LDT), respectivamente. Estos registros, de 8 bytes de tamaño (64 bits), permiten apuntar a cualquier dirección a lo largo de toda la memoria del sistema.

7. Arquitectura Combinada: Segmentación y Paginación

En los sistemas operativos modernos, no se trabaja de forma puramente segmentada o puramente paginada de forma aislada, sino que se emplea un esquema que combina ambas arquitecturas simultáneamente. Este enfoque fue popularizado por los microprocesadores Intel (como el x86 y los Pentium) y aprovechado por sistemas como Linux.

El proceso inicia segmentando el programa: su código, datos y pila se dividen en diferentes segmentos. Una vez definidos, en lugar de mapearse directamente a memoria física, cada uno de estos segmentos ingresa a la unidad de paginación. La paginación toma estos segmentos y los divide en páginas estándar. Las páginas resultantes se distribuyen entonces en los marcos de página de la memoria física, pudiendo quedar dispuestas de forma no consecutiva. Aunque este método genera cierto grado de fragmentación interna debido a que el segmento final de un archivo puede no llenar la totalidad de la página, sigue siendo una forma muy eficiente de uso de la memoria.

8. Flujo de Traducción en el Sistema Combinado

El flujo operativo completo funciona atravesando tres etapas lógicas para llegar al dato en el hardware:

1. **De Dirección Lógica a Lineal:** El programa proporciona una dirección lógica (Selector + Desplazamiento), que ingresa a la unidad de segmentación para devolver una **dirección lineal** de 32 bits.
2. **De Dirección Lineal a Física:** Esta dirección lineal es tratada como una dirección virtual que alimenta a la unidad de paginación, encargada finalmente de convertirla a una dirección física real.

Antes de que este flujo funcione, el microprocesador debe ser configurado desde su modo real inicial hacia el Modo Protegido, activando los bits de configuración de paginación y segmentación del sistema.

Paso a paso técnico del hardware:

- Se carga un registro interno de CPU (por ejemplo, CS para código) con el selector de segmento de 16 bits.
- El hardware interpreta el bit TI (para elegir la tabla GDT o LDT) y el índice.
- Se localiza la tabla leyendo los registros base (GDTR o LDTR).
- Se extrae el descriptor del segmento correspondiente (64 bits).
- Se verifica estrictamente que el desplazamiento (de hasta 20 bits) sea menor que el límite. Si es superior, se lanza una excepción (fallo de segmentación).
- Pasada la verificación, se produce la dirección lineal de 32 bits.
- Esta dirección lineal ingresa al proceso de paginación de dos niveles. Por ejemplo, se particionan los 32 bits: 12 bits para el desplazamiento en páginas de 4KB, y el resto para índices de directorios y tablas de páginas.
- Haciendo uso de registros como el CR3 (que indica el inicio del directorio de páginas), se navegan las tablas de paginación hasta conseguir la dirección base del marco.
- Se suma el desplazamiento final de 12 bits para obtener la dirección física en memoria.

9. Costos Administrativos y Límites del Sistema

Esta arquitectura tiene límites bien definidos. El sistema soporta un máximo de 16,384 (16K) segmentos por cada proceso individual. La razón matemática tras esto radica en el selector de 16 bits: 3 bits se reservan para fines administrativos (TI y RPL), dejando 13 bits para crear el índice. Como 2^{13} equivale a 8,192 (8K) descriptores, y existen dos tablas (GDT y LDT), se suman $8K + 8K = 16K$ descriptores o segmentos posibles por proceso.

Dado que cada descriptor pesa 8 bytes, una tabla completa de descriptores ocupará unos 64 Kilobytes (64KB). Por lo tanto, la administración estructural introduce un requerimiento de memoria RAM adicional que representa una "pérdida" de espacio utilizable. Por ejemplo, un

proceso simple de 1 MegaByte requerirá destinar esos 64KB extras solo para mantener su tabla de descriptores, además de lo que necesiten sus tablas de paginación.

Adicionalmente, hay un coste de rendimiento: para acceder a un simple valor en memoria, la CPU debe hacer múltiples lecturas y saltos internos para decodificar los registros y las diferentes tablas (descriptores y páginas), lo cual retrasa ligeramente la ejecución general frente a un modelo completamente plano.

10. Perspectiva Histórica

Antes de que existieran Unidades de Manejo de Memoria (MMU) que gestionaran la segmentación por hardware, los programadores administraban y dividían sus códigos, datos y pilas en segmentos manuales ubicados en zonas fijas y distintas de la memoria. La segmentación moderna integrada por hardware tuvo su auge e inclusión fundamental a partir de la llegada del procesador Intel 80386 (386). Los modelos predecesores (como el Intel 80286) ofrecían versiones más primitivas que no habilitaban la segmentación y paginación conjuntas de la misma manera, requiriendo operaciones manuales bajo modo real. Hoy en día, los procesadores inician en un "modo real" limpio y, posteriormente, el sistema operativo activa el "modo protegido" para delegar estas tareas administrativas al procesador, volviendo el sistema más seguro, estable y fácil de usar para los desarrolladores.